L5-6 讲义：智能体设计与实现

课程: MBA 大模型智能体课程
课时: 第 5-6 讲（90 分钟）
适用: 讲师备课 / 学生预习与复习
对应 Slides: `slides-marp/day1-lesson5-6-agent.md`

本讲核心问题

1. **什么是 AI Agent？它和普通聊天机器人有什么区别？**
2. **Agent 如何"思考"？ ReAct 框架的核心逻辑是什么？**
3. **Agent 如何自我改进？反思机制如何工作？**
4. **如何设计一个 Agent？从需求到落地的完整流程**

为什么这节课是核心？

如果说前面几节课讲的是"如何让 AI 回答问题"，这节课讲的是"如何让 AI 完成任务"。

这是一个根本性的跨越：

维度	聊天机器人	AI Agent
交互模式	问答式	任务式
决策能力	被动响应	主动规划
工具使用	无/有限	丰富且动态
任务范围	单轮完成	多步骤复杂任务
错误处理	简单重试	反思改进

> **MBA 视角**：Agent 是 AI 从"工具"变成"员工"的关键技术。理解 Agent，就理解了 AI 如何在组织中承担实际工作。

Part 1：什么是 AI Agent？

定义

AI Agent 是一个**能自主决策、使用工具、完成复杂任务**的 AI 系统。

用一个类比来理解：

普通 LLM = 百科全书 - 你问什么，它查什么 - 不会主动做事

AI Agent = 远程员工 - 你给任务，它自己规划怎么做 - 需要时自己查资料、用工具 - 遇到问题自己想办法

Agent 架构总览

Agent 架构总览 来源: Lilian Weng, "LLM Powered Autonomous Agents"(2023)

上图展示了 Agent 的核心架构：- **LLM** 作为"大脑"进行规划和推理 - **Memory** 提供短期和长期记忆能力 - **Tools** 让 Agent 能与外部世界交互 - **Planning** 让 Agent 能分解复杂任务

Agent 的四要素

就像一个员工需要具备的能力：

要素	能力	商业类比
感知 (Perceive)	理解用户意图、获取环境信息	理解老板需求
思考 (Think)	分析问题、制定方案	制定工作计划
行动 (Act)	调用工具、执行操作	执行具体任务
反思 (Reflect)	评估结果、改进策略	复盘总结经验

Agent 的核心循环

Agent 不是一次性输出，而是**循环执行**直到任务完成：

感知 → 思考 → 行动 → 观察结果 → 反思 → 继续思考 → ...

这个循环会一直进行，直到：- 任务完成 - 达到最大步数 - 用户中断

Part 2：ReAct 框架

论文背景

ReAct (Reasoning + Acting) 是 2022 年 Google Research 和 Princeton 提出的 Agent 框架，将**推理**和**行动**交织进行。

ReAct 执行示例

ReAct 框架示例 来源: Lilian Weng, "LLM Powered Autonomous Agents"(2023)

上图展示了 ReAct 的实际执行过程：- **Thought**：Agent 的推理过程 - **Action**：Agent 调用的工具 - **Observation**：工具返回的结果

核心思想

传统方法把思考和行动分开：

思考 → 思考 → 思考 → 行动

ReAct 把它们交织在一起：

思考 → 行动 → 观察 → 思考 → 行动 → 观察 → ...

为什么这样更好?

想象你在做市场调研：

传统方法（先想完再做）：

```
1. 我觉得市场规模大概是100亿
2. 我觉得主要竞争对手是A、B、C
3. 我觉得增长率大概是15%
4. 开始写报告...
```

问题：全凭猜测，可能全错

ReAct 方法（边做边想）：

```
1. 我需要查市场规模 → 搜索"中国咖啡市场规模2026"
2. 观察：搜索结果显示市场规模是850亿
3. 我需要查竞争格局 → 搜索"中国咖啡连锁品牌排名"
4. 观察：瑞幸第一、星巴克第二...
5. 基于真实数据写报告
```

优势：每一步都基于真实信息

实际执行示例

用户任务："帮我查一下苹果公司最新的市值，并与微软比较"

Agent执行过程：

```
Thought 1: 我需要查询苹果公司的当前市值
Action 1: search("Apple Inc market cap 2026")
Observation 1: Apple Inc. (AAPL) has a market cap of $3.2 trillion...

Thought 2: 现在我需要查询微软的市值
Action 2: search("Microsoft market cap 2026")
Observation 2: Microsoft Corporation (MSFT) market cap is $3.4 trillion...

Thought 3: 我已经有了两家公司的数据，可以进行比较了
Action 3: 生成比较分析

Final Answer:
截至2026年2月，苹果市值约3.2万亿美元，微软市值约3.4万亿美元。
微软市值超过苹果约2000亿美元，差距约6%...
```

ReAct 的优势

优势	说明
透明性	每一步推理可追溯，出问题知道在哪
可控性	可以中途干预、调整方向
准确性	基于实际观察而非臆测
可调试	出错时容易定位问题

Part 3：反思机制

为什么需要反思?

Agent 执行过程中可能出错：- 工具调用失败 - 获取的信息不准确 - 推理逻辑有漏洞 - 结果不符合预期

没有反思的 Agent 就像一个不会总结经验的员工——同样的错误会反复犯。

Reflexion 机制

Reflexion 反思机制 来源: Lilian Weng, "LLM Powered Autonomous Agents"(2023)

上图展示了 Reflexion 的工作原理：- Agent 执行任务后评估结果 - 如果失败，生成反思摘要 - 反思存入记忆，指导下次执行

三种反思策略

1. Self-Refine（自我改进）

生成初稿 → 自我评估 → 改进 → 再评估 → ...

示例：
初稿："市场规模100亿"
评估："没有数据来源，可信度低"
改进："根据艾瑞咨询报告，2025年市场规模为95亿元[1]"

优点：简单、成本低 **缺点**：可能陷入局部最优（自己看不到自己的问题）

2. Reflexion（反思记忆）

失败经验 → 提取教训 → 存入记忆 → 下次避免

示例：
失败："搜索'苹果市值'返回了水果苹果的结果"
教训："搜索公司信息时要加'公司'或股票代码"
记忆：存储这条教训
下次：搜索"苹果公司AAPL市值"

优点：持续学习、避免重复错误 **缺点**：需要维护记忆系统

3. Critic Agent（批评者 Agent）

执行Agent完成任务 → Critic Agent评估 → 反馈改进建议

类似于：
- 员工写报告 → 主管审核 → 修改意见 → 员工修改

优点：视角独立、更客观 **缺点**：成本翻倍、需要协调两个 Agent

如何选择反思策略？

场景	推荐策略	原因
简单任务	Self-Refine	成本低，够用
需要积累经验	Reflexion	持续改进
高风险决策	Critic Agent	需要双重检验
成本敏感	Self-Refine	最便宜

Part 4：规划算法

为什么需要规划？

复杂任务需要分解成子任务，好的规划能够：- 提高执行效率 - 减少无效尝试 - 处理任务依赖

Tree of Thoughts (ToT)

ToT 的核心思想：**不要只走一条路，同时探索多条路**

问题

```
        思路A      思路B      思路C

                   （剪枝）

      A1    A2  B1    B2
       ↓         ↓
     评估       评估
       ↓
    选择最优
```

示例：解决一个商业问题

问题：如何提升用户留存？

思路A：优化产品体验
  A1：简化注册流程
  A2：增加新手引导

思路B：增强用户粘性
  B1：增加社交功能
  B2：设计积分体系

思路C：提高服务质量
  （评估后认为不是核心问题，剪枝）

评估每个方案的可行性和ROI，选择最优

规划方法选择建议

任务复杂度	推荐方法	说明
简单	直接执行	ReAct 足够
中等	ToT	关键决策点展开多路径
复杂	分层规划	先粗后细
超复杂	人机协作	Agent 提建议，人做决策

Part 5：Agent 设计模式

模式 1：单一 Agent

用户 → Agent → 结果

适用：简单任务、快速响应

模式 2：工具增强 Agent

用户 → Agent ↔ 工具库 → 结果

工具库包括：
- 搜索引擎
- 计算器
- 数据库
- API调用

- 代码执行

模式 3：多 Agent 协作

```
用户 → Coordinator Agent


    ↓          ↓          ↓
 Agent A    Agent B    Agent C


              ↓
           结果汇总
```

适用：复杂任务需要多种专业能力

设计模式决策树

```
任务复杂度如何?

        简单（单步可完成）
            单一Agent，无需工具

        中等（需要外部信息/计算）
            工具增强Agent

        复杂（需要多种专业能力）

            是否可以并行?

                是 → 多Agent并行
                否 → 多Agent串行
```

Part 6：前沿 Agent 框架与产品

编码 Agent

产品	特点	适用场景
Devin	全自主开发、端到端交付	工程外包
Claude Code	CLI 工具、CLAUDE.md 配置、Thinking 模式	开发者日常
Codex CLI	GitHub 深度集成、Web+CLI 双端	企业开发

AI IDE

产品	特点	适用场景
Cursor	Composer 多文件编辑、Background Agent、Plugins	项目开发
Windsurf	Cascade 引擎、持久记忆	快速原型

通用 Agent 框架

产品	特点	适用场景
OpenClaw	多渠道、Skills 系统、MCP 集成、开源	个人 AI 助手
Manus	Context Engineering、被 Meta 收购	复杂任务

企业 Agent

产品	特点	适用场景
Claude Cowork	Office 集成、企业 Plugins	企业办公

框架选型建议

需求	推荐
个人开发助手	Claude Code / Cursor
企业办公自动化	Claude Cowork
构建自定义 Agent	OpenClaw（开源）
端到端开发外包	Devin
快速原型	Windsurf

Part 7：企业应用案例

案例 1：AI 投研助手

背景：某券商需要处理海量研报、新闻、财务数据

Agent 架构：

```
PM Agent（项目经理）

    Data Agent（数据收集）
        搜索、爬取、数据库查询

    Analyst Agent（分析研究）
        财务分析、竞争格局

    Writer Agent（报告撰写）
        结构化输出

    Review Agent（合规审核）
         风险检查、合规审核
```

效果：- 研报初稿时间：4 小时 → 30 分钟 - 数据覆盖：+200% - 合规问题：-80%

案例 2：Klarna 客服 AI

背景：瑞典金融科技公司，每月数百万客服请求

方案：基于 OpenAI 的客服 Agent，处理退款、查询、投诉等场景

效果（2024 年数据）：- 处理量：相当于 700 名全职客服 - 解决时间：11 分钟 → 2 分钟 - 客户满意度：持平（关键！）- 年节省成本：$4000 万

争议与反思：- 引发裁员担忧 - “效率提升”vs”就业影响”的平衡 - 适合重复性高、标准化强的场景

Part 8：Agent 开发实践

30/60/90 天路线图

第一阶段（0-30 天）：验证可行性

- ☐ 选择 1 个具体业务场景
- ☐ 用现成工具（如 Coze）搭建原型
- ☐ 收集 100 个真实用户反馈
- ☐ 计算粗略 ROI

第二阶段（30-60 天）：小规模试点

- ☐ 优化 Prompt 和工作流
- ☐ 接入必要的内部系统
- ☐ 建立评估指标体系
- ☐ 培训试点用户

第三阶段（60-90 天）：规模化落地

- ☐ 完善异常处理
- ☐ 建立监控告警
- ☐ 制定 SOP 和培训材料
- ☐ 扩展到更多用户/场景

常见失败原因

失败模式	根因	解决方案
Agent 不知道该做什么	任务描述模糊	明确成功标准
Agent 做了不该做的事	权限控制不足	最小权限原则
Agent 陷入死循环	没有终止条件	设置最大步数
Agent 输出不稳定	缺少评估机制	建立质量门控
成本失控	没有预算限制	设置 token 上限

课堂实操

设计一个 Agent（15 分钟）

选择以下场景之一，设计 Agent 架构：

1. **客服 Agent**：处理退款申请
2. **研究 Agent**：竞争对手分析
3. **招聘 Agent**：简历初筛
4. **内容 Agent**：社交媒体日更

需要回答：- 需要什么工具？- 采用什么设计模式？- 如何处理失败？- 如何评估效果？

本讲总结

核心概念

- **Agent = 能自主决策、使用工具、完成复杂任务的 AI 系统**
- **ReAct = 推理和行动交织进行**
- **反思 = Agent 自我评估和改进的能力**
- **规划 = 将复杂任务分解为可执行步骤**

Agent vs 传统 AI 的本质区别

传统 AI	Agent
回答问题	完成任务
一次性输出	循环执行
被动响应	主动规划
固定能力	工具扩展

MBA 关键洞察

1. **Agent 是 AI 从"工具"变成"员工"的技术**
2. **设计 Agent = 设计一个虚拟岗位**
3. **Agent 需要监督，就像新员工需要管理**
4. **从简单场景开始，逐步扩展**

延伸阅读

论文

- ReAct: Synergizing Reasoning and Acting
- Reflexion: Language Agents with Verbal Reinforcement Learning
- Tree of Thoughts: Deliberate Problem Solving

框架文档

- LangChain Agents
- OpenClaw Documentation
- AutoGen

博客

- Anthropic: Building Effective Agents
- Manus: Context Engineering for AI Agents

本讲义最后更新：2026-02-28